

5. WebMvcConfigurer를 통한 인터셉터 등록과 경로패턴 설정

필터가 서블릿 컨테이너 수준에서 요청을 가로챌다면, 인터셉터 (`HandlerInterceptor`) 는 Spring MVC 내부, 즉 `DispatcherServlet` 이 핸들러를 찾은 이후에 요청을 가로챈다. 이번 글에서는 인터셉터를 등록하는 방법과 Spring Boot 에서의 주의점, 그리고 Java 설정과 XML 설정의 적용 범위 차이를 정리한다.

들어가기 전에 : HandlerInterceptor

`HandlerInterceptor` 는 세 개의 콜백을 제공한다.

메서드	호출 시점	비고
<code>preHandle()</code>	핸들러(컨트롤러) 실행 전	<code>false</code> 를 반환하면 이후 체인과 핸들러 실행을 중단한다
<code>postHandle()</code>	핸들러 실행 후, 뷰 렌더링 전	<code>@ResponseBody</code> / <code>ResponseEntity</code> 는 이 시점에 이미 응답이 쓰여 있을 수 있다
<code>afterCompletion()</code>	요청 처리가 완전히 끝난 후	예외 발생 여부와 관계없이 호출 (<code>preHandle</code> 이 <code>true</code> 였던 경우)

필터와 달리 인터셉터는 어떤 핸들러가 선택됐는지 알 수 있다는 점이 핵심이다.

5.1 기본 등록 방법 : WebMvcConfigurer

들어오는 요청에 적용할 인터셉터는 다음과 같이 등록한다.

```

@Configuration
public class WebConfiguration implements
WebMvcConfigurer {

    @Override
    public void addInterceptors(InterceptorRegistry
registry){
        registry.addInterceptor(
            new LocaleChangeInterceptor()
        );
        registry.addInterceptor(
            new UserRoleAuthorizationInterceptor()
        ).addPathPatterns("/**").excludePathPatterns("/admin/**
");
    }
}

```

WebMvcConfigurer 인터페이스를 구현하고 `addInterceptors()` 를 오버라이딩한다. 그리고 `registry.addInterceptor(...)` 메서드로 인터셉터를 추가한다.

여기서 다음과 같은 등록 패턴을 볼 수 있다.

- `LocaleChangeInterceptor` 처럼 경로 지정 없이 등록하면 모든 요청(모든 핸들러)에 적용된다.
- `addPathPatterns` 와 `excludePathPatterns` 를 체이닝하면 적용 대상을 세밀하게 제어할 수 있다.
- 예제에서는 `/**` 로 모든 경로에 적용하면서 `/admin/**` 는 제외하였다.

- 인터셉터는 **등록한 순서대로** 실행된다. 명시적으로 순서를 지정하려면 `.order(int)` 를 쓴다.
- `preHandle()` 은 등록 순서대로, `postHandle()` / `afterCompletion()` 은 **역순으로** 호출된다.
- 위 예제처럼 `new` 로 직접 생성하면 인터셉터는 스프링 빈이 아니다. 인터셉터 안에서 다른 빈을 주입받아야 한다면, 인터셉터를 빈으로 등록한 뒤 설정 클래스에서 주입받아 `addInterceptor()` 에 넘기면 된다.

5.2 MVC 설정을 켜는 방법과 Spring Boot 에서의 주의점

해당 설정이 동작하려면 **MVC 설정이 활성화되어** 있어야 한다. 순수 Spring MVC 환경에서는 `@EnableWebMvc` 를 사용한다.

```
@Configuration
@EnableWebMvc
public class WebConfiguration implements
WebMvcConfigurer {
    ...
}
```

하지만 **Spring Boot** 를 사용한다면 여기서 주의사항이 있다. 공식 문서에서는 Spring Boot 사용 시 `WebMvcConfigurer` 타입의 `@Configuration` 클래스를 사용하되 `@EnableWebMvc` 는 붙이지 **않을** 것을 권한다.

Spring Boot 프로젝트에서 인터셉터를 추가하려고 `@EnableWebMvc` 를 붙이면 **Boot** 의 **MVC** 자동 구성이 꺼지면서, 정적 리소스 처리

(`classpath:/static` 등)나 메시지 컨버터 설정
(`spring.jackson.*` 등)이 갑자기 동작하지 않게 된다.

왜 꺼질

까? `@EnableWebMvc` 는 `DelegatingWebMvcConfiguration (= WebMvcConfigurationSupport)` 을 빈으로 등록한다. 그런데 Boot 의 `WebMvcAutoConfiguration` 은 `@ConditionalOnMissingBean(WebMvcConfigurationSupport.class)` 조건이 걸려 있어서, 이 빈이 존재하면 자동 구성 전체가 물러난다. 즉, `@EnableWebMvc` 는 "MVC 설정을 내가 전부 직접 하겠다"는 선언과 같다.

Boot에서는 `WebMvcConfigurer` 구현만 하고 `@EnableWebMvc` 는 생략하는 것이 정답이다.

5.3 Java 설정과 XML 설정의 적용 범위 차이 : MappedInterceptor

XML 설정은 인터셉터를 `MappedInterceptor` 빈으로 선언하며, 이 빈은 다른 프레임워크에서 온 것들을 포함한 모든 `HandlerMapping` 빈에 의해 감지된다. 반면 Java 설정은 인터셉터를 자신이 관리하는 `HandlerMapping` 빈에만 전달한다.

즉, Java 설정은 범위가 좁고 명시적이고, XML 방식은 더 넓게 퍼진다.

방식	인터셉터 전달 대상
XML (<code><mvc:interceptors></code>)	<code>MappedInterceptor</code> 빈 → 컨텍스트의 모든 <code>HandlerMapping</code> 이 감지
Java (<code>addInterceptors()</code>)	MVC Java 설정이 만든 <code>HandlerMapping</code> 에만 직접 전달

그래서 MVC Java 설정을 쓰면서 **동일한 인터셉터를 Spring MVC 와 다른 프레임워크의 `HandlerMapping` 빈에서 함께 재사용**하려면, 문서에 서는 두 가지 방법을 제시한다.

1. `MappedInterceptor` 빈으로 선언하고, Java 설정 (`addInterceptors()`)에서는 수동으로 추가하지 않는다.
2. `MappedInterceptor` 빈 없이, **Java 설정과 다 른 `HandlerMapping` 빈 양쪽에 같은 인터셉터를 각각 설정**한다.

두 방법 중 하나만 선택해야 한다. `MappedInterceptor` 빈으로 선언하면서 Java 설정에도 추가하면, MVC 의 `HandlerMapping` 입 장에서는 같은 인터셉터를 두 번 받게 되어 **중복 적용**된다.

1번 방식은 다음과 같이 작성할 수 있다.

```
@Bean
public MappedInterceptor roleInterceptor() {
    return new MappedInterceptor(
        new String[] {"/**"},          //
includePatterns
        new String[] {"/admin/**"},    //
excludePatterns
        new UserRoleAuthorizationInterceptor()
    );
}
```

5.4 마무리

인터셉터는 **보안 계층으로 적합하지 않다**. 어노테이션 기반 컨트롤러의 경로 매칭은 끝 슬래시나 경로 확장자 등을 **투명하게 매칭**할 수 있는데, 인터셉터의 경로 패턴은 이와 다르게 판단할 수 있어 **불일치**가 생길 수 있기 때문이다.

예를 들어 인터셉터는 `/admin` 만 검사하도록 설정했는데, 컨트롤러 매핑이 `/admin/` 요청까지 같은 핸들러로 처리한다면 검사를 우회하는 틈이 생긴다. (Spring 6.0 부터 끝 슬래시 매칭이 기본으로 꺼져 위험이 줄었지만, 매칭 옵션에 따라 여전히 불일치가 생길 수 있다.)

그래서 공식 문서는 **Spring Security** 나 서블릿 필터 체인에 통합된 방식을 가능한 한 앞단에 적용하기를 권장한다. (앞서 다룬 `UrlHandlerFilter` 로 경로를 미리 정규화해 두는 것도 이런 불일치를 줄이는 데 도움이 된다.)

정리하면

- 인터셉터는 `WebMvcConfigurer` 의 `addInterceptors()` 에 서 `InterceptorRegistry` 에 등록한다.
- `addPathPatterns()`, `excludePathPatterns()` 로 적용 범위를 조절한다.
- Spring Boot 에서는 `@EnableWebMvc` 를 붙이지 않는다.
- 인증·인가 같은 보안 로직은 인터셉터가 아니라 **필터 계층(Spring Security)** 에 둔다.